Следствие 1 *Свързаният граф G притежава Ойлеров път с различни начало и край тогава и само тогава, когато в G има точно два върха $v \in V$ с нечетна степен $d(v)$.*

Доказателство. ($\Longrightarrow$) Нека в G има Ойлеров път от v_1 до v_2, където $v_1 \neq v_2$. Този път изглежда така:

$$v_1 - \ldots\ldots - v - \ldots\ldots - v_2.$$

Всички върхове $v \in V$, такива че $v \neq v_1, v_2$ имат четна степен, тъй като броят на ребрата с край v е два пъти броя на участията на v в пътя. Върховете v_1 и v_2 имат нечетна степен, тъй като точно едно от участията им в пътя допринася с едно ребро към степента им.

($\Longleftarrow$) Нека v_1 и v_2 са двата върха на G с нечетна степен. Добавяме нов връх u към графа G и две нови ребра $\{u, v_1\}, \{u, v_2\}$. Ясно е, че получаваме свързан граф, в който всеки връх има четна степен: върховете на G, различни от v_1 и v_2 запазват степента си, степента на v_1 и v_2 се повишава с 1 и става четна, степента на новия връх u е 2. От теоремата на Ойлер, в новия граф съществува Ойлеров цикъл. В него новите ребра $\{u, v_1\}$ и $\{u, v_2\}$ трябва да участват точно по веднъж. Но това може да стане единствено, ако цикълът съдържа (еднократно) последователността v_1, u, v_2 или v_2, u, v_1. Премахваме върха u и двете ребра, свързани с него и така получаваме Ойлеров път от v_1 до v_2 (или от v_2 до v_1).

Теоремата на Ойлер и следствието от нея имат аналози и за ориентирани мултиграфи.

Дефиниция 2 *Нека $G = (V, E, f_G)$ е краен ориентиран мултиграф.* Ойлеров маршрут *в G е такъв, който минава през всяко ребро на G точно по веднъж. Ако началото и края на този маршрут съвпадат, то го наричаме* Ойлеров контур. *Казваме, че G е* Ойлеров мултиграф, *ако в G съществува Ойлеров контур.*

Теорема 3 (Ойлер, ориентирани мултиграфи) *Крайният ориентиран слабо свързан мултиграф $G = (V, E, f_G)$ е Ойлеров тогава и само тогава, когато полустепените на входа и изхода на всеки връх съвпадат, т.е. $d^-(v) = d^+(v)$ за всеки $v \in V$.*

Доказателство. Аналогично на теоремата на Ойлер за неориентирани графи. Ако в G има Ойлеров контур, то всяко участие на връх $v \in V$ в него допринася с 1 към полустепента на входа $d^-(v)$ и на изхода $d^+(v)$. Така полустепените на входа и изхода на всеки връх $v \in V$ съвпадат с броя на участията на v в Ойлеровия контур.

Обратно, ако полустепените на входа и изхода на всеки връх съвпадат, то строим Ойлеров контур в G буквално със същия алгоритъм, както в неориентирания случай.

Следствие 4 *Крайният ориентиран слабо свързан мултиграф* $G = (V, E, f_G)$ *притежава Ойлеров маршрут с различни начало и край тогава и само тогава, когато има два върха* v_1, v_2*, такива че*

$$d^+(v_1) = d^-(v_1) + 1,\ d^-(v_2) = d^+(v_2) + 1$$

и $d^-(v) = d^+(v)$ *за всеки* $v \in V \setminus \{v_1, v_2\}$.

Доказателство. Тук не се налага добавяне на нов връх, достатъчно е да добавим само ново ребро от v_2 към v_1. Избираме Ойлеров контур в получения мултиграф и премахваме от него добавеното ребро. Получаваме Ойлеров маршрут от v_1 до v_2 в оригиналния мултиграф G.

Хамилтонови обхождания

Дефиниция 5 *Нека* $G = (V, E)$ *е неориентиран свързан граф.*

Хамилтонов път *в* G *наричаме такъв път, който минава през всеки връх точно по веднъж. Ако началото и края на този път съвпадат, говорим за* Хамилтонов цикъл. *Графът* G *е* Хамилтонов*, ако в него съществува поне един Хамилтонов цикъл.*

Не е известен ефективен алгоритъм, който да проверява дали даден граф е Хамилтонов. Това е свързано с факта, че нямаме добра характеризация на Хамилтоновите графи, подобна на Ойлеровите графи. Всъщност има редица от *достатъчни* условия (но не и необходими) един граф да е Хамилтонов, например да съдържа поне 3 върха и степента на всеки връх да е поне половината от броя на всички върхове.

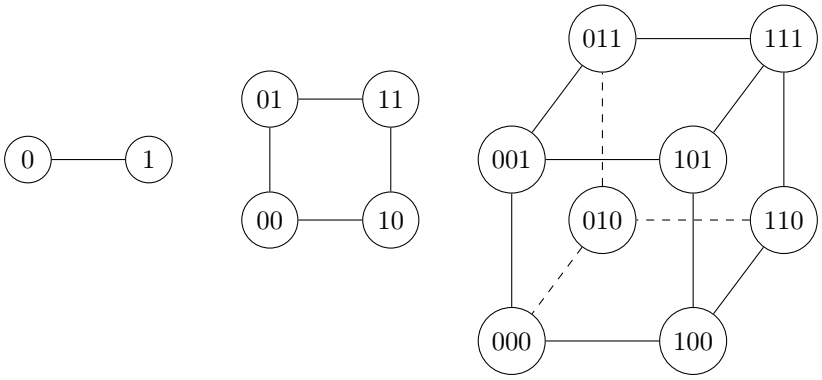
Ще завършим тази секция с пример за безкрайна редица от Хамилтонови графи.

Дефиниция 6 *Графът* $\mathcal{B}_n = (\{0,1\}^n, E_n)$*, където*

$$E_n = \{\{\alpha, \beta\} \mid \alpha, \beta \in \{0,1\}^n \text{ се различават в точно една позиция}\}$$

ще наричаме n-мерен двоичен куб *(съответно върховете му наричаме* n*-мерни двоични вектори).*

Например, при $n = 1, 2, 3$ получаваме следните графи:



Твърдение 7 *При $n \geq 2$ графът $\mathcal{B}_n$ е Хамилтонов.*

Доказателство. Индукция по n.

База: при $n = 2$ имаме Хамилтонов цикъл $00, 01, 11, 10, 00$.

Стъпка: нека $\alpha_0, \alpha_1, \ldots, \alpha_{2^n-1}, \alpha_0$ е Хамилтонов цикъл в $\mathcal{B}_n$. Да разгледаме следната последователност от върхове в $\mathcal{B}_{n+1}$:

$$0\alpha_0, 0\alpha_1, \ldots, 0\alpha_{2^n-1}, 1\alpha_{2^n-1}, 1\alpha_{2^n-2}, \ldots, 1\alpha_0, 0\alpha_0.$$

Лесно се вижда, че всеки два съседни върха се различават точно в една позиция, т.е. това е цикъл в $\mathcal{B}_{n+1}$ и освен това всеки връх от $\{0,1\}^{n+1}$ (с изключение на началото) участва еднократно. Казано по друг начин, това е Хамилтонов цикъл в $\mathcal{B}_{n+1}$.

Накрая да отбележим, че при $n = 1$ в $\mathcal{B}_1$ няма Хамилтонов цикъл, но има Хамилтонов път.

Рекурсивни процедури за обхождане в дълбочина

Ще дадем псевдокод на рекурсивни програмни реализации на обхождане в дълбочина.

Считаме, че е зададен (ориентиран) граф $G = (V, E)$ със списък на съседите: нека $adj[v]$ да е списък на всички върхове w, такива че има ребро от v до w.

```
procedure DFS(v)
  marked[v] = true; // маркираме v за обходен
  for all u ∈ adj[v]
    if (not marked[u])
      {D = D ∪ {{v, u}}; // {D = D ∪ {(v, u)}; за ориентиран граф
        DFS(u); }
  print(v);
```

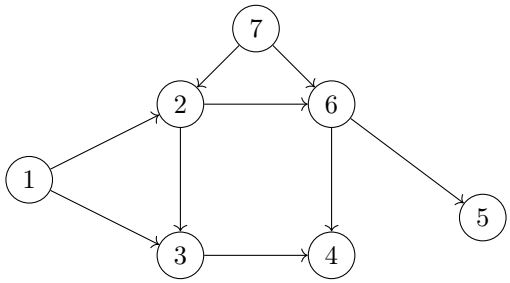
Забележете, че принтираме v чак след като приключи for-цикъла.

За да извикаме тази процедура, използваме следния главен псевдокод:

```
for all r ∈ V
  if (not marked[r])
    DFS(r);
```

В множеството D запазваме ребрата, по които върви обхождането. Ако G е неориентиран, след изпълнението D ще съдържа ребрата на покриващи дървета на всички компоненти на свързаност на G.

Да проиграем процедурата върху следния ориентиран граф:



Извикванията се извършват в следната последователност:

$$DFS(1) \to DFS(2) \to DFS(3) \to DFS(4).$$

При изпълнение на $DFS(4)$ няма ново рекурсивно извикване, така че се изпълнява $print(4)$. След това изпълнението се връща към $DFS(3)$, където for-цикълът приключва и се изпълнява $print(3)$. Връщаме се в

$$DFS(2) \to DFS(6) \to DFS(5),$$

след което се извикват последователно $print(5), print(6), print(2), print(1)$. С това извикването $DFS(1)$ приключва. Тъй като имаме още един немаркиран връх, то ще последва още една итерация с извикване $DFS(7)$, която ще извика $print(7)$.

Още веднъж да изброим редът на принтиране на върховете:

$$4, 3, 5, 6, 2, 1, 7.$$

Тази редица има свойството, че всички ребра в графа сочат отдясно наляво, т.е. тя представлява топологическо сортиране на графа в обратен ред. С други думи, ако графът представя частична наредба, то преобърнатата редица дефинира линейна наредба, която разширява частичната наредба.

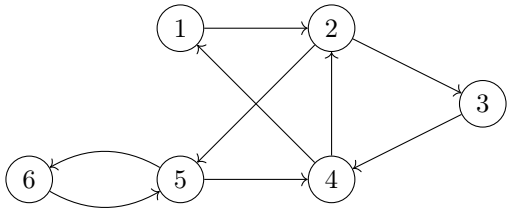
Следващата процедура е подобна на обхождането в дълбочина и чрез нея се построява Ойлеров цикъл (контур).

```
procedure DFS_E(v)
 for all u ∈ adj[v]
  { remove u from adj[v];
    remove v from adj[u]; // само за неориентиран граф
    DFS_E(u); }
 print(v);
```

Процедурата е базирана на алгоритъма от доказателството на теоремата на Ойлер: вървим по необходени ребра, докато е възможно, след което

се връщаме до връх, от който има необходено ребро и продължаваме рекурсивно по същия начин. Главното извикване на процедурата е DFS_E(v), където v е произволен начален връх.

Като пример да разгледаме следния ориентиран граф:



Предполагаме, че списъците на съседите са подредени в нарастващ ред на номерата на върховете.

При начален връх 1 последователността от извиквания започва така:

$$DFS_E(1) \to DFS_E(2) \to DFS_E(3) \to DFS_E(4) \to DFS_E(1),$$

като в последното извикване $DFS_E(1)$ няма повече необходени ребра с начало 1 и затова се изпълнява $print(1)$. По-нататък се връщаме към извикването $DFS_E(4)$, което води до извикванията

$$DFS_E(4) \to DFS_E(2) \to DFS_E(5) \to DFS_E(4).$$

Следва $print(4)$ и връщане към $DFS_E(5)$, което води до

$$DFS_E(5) \to DFS_E(6) \to DFS_E(5).$$

В този момент всички ребра вече са обходени, така че имаме само връщания, при които се изпълняват принтиранията: $print(5)$, $print(6)$, $print(5)$ (тук приключва разклонението от $DFS_E(5)$), $print(2)$, $print(4)$ (тук приключва $DFS_E(4)$), $print(3)$, $print(2)$, $print(1)$.

Принтираната последователност $1, 4, 5, 6, 5, 2, 4, 3, 2, 1$ след преобръщане представлява Ойлеров контур в дадения граф.

Минимални пътища в графи с тегла по ребрата

Нека $G = (V, E)$ е свързан граф и да предположим, че е зададена функция $c : E \to \mathbb{R}$. За всяко ребро $e \in E$ реалното число $c(e)$ ще наричаме *цена или тегло* на реброто e.

При даден път $p = v_0, v_1, \ldots, v_k$, *цена или тегло на пътя* p наричаме реалното число $c(p) = \sum_{j<k} c(\{v_j, v_{j+1}\})$, т.е. сумата от цените/теглата на ребрата, участващи в пътя.

За целите на тази секция ще предполагаме, че $c(e) \geq 0$ за всяко ребро e.

Дефинираме функцията $dist : V \times V \to \mathbb{R}^{\geq 0}$ по следния начин:

$$dist(u, v) \ = \ \min\{c(p) \mid p \text{ е път с начало } u \text{ и край } v\}.$$

По този начин, $dist(u, v)$ е най-малката възможна цена на път с начало u и край v. Път p от u до v, такъв че $c(p) = dist(u, v)$ наричаме *минимален път* от u до v.

Забележете, че за всеки два върха $u, v \in V$ съществува *прост* минимален път от u до v. Един минимален път може да не е прост, единствено ако съдържа цикъл с цена 0 и този цикъл може да бъде премахнат. От това следва, че $dist$ е добре дефинирана функция, тъй като простите пътища от u до v са краен брой.

Ако в графа допускахме ребра с отрицателни тегла, то функцията $dist$ нямаше да е добре дефинирана. Причината е, че в такъв случай между никои два върха няма път с минимална цена: можем да вървим последователно напред и назад по някое отрицателно ребро (и то е достижимо от всички върхове), като така ще получаваме все по-малка цена на пътя.

Ако всички ребра в графа имат цена 1, то функцията $dist$ е същата, която разгледахме при обхождането в ширина.

Отново важи *принципа за оптималност*: ако p е минимален (прост) път, то всяка част от него също е минимален (прост) път.

Също е вярно *неравенството на триъгълника*:

$$dist(u, v) \leq dist(u, w) + dist(w, v).$$

Като следствие от това неравенство, ако има ребро от w до v, то

$$dist(u, v) \leq dist(u, w) + c(\{w, v\}), \qquad (*)$$

тъй като $dist(w, v) \leq c(\{w, v\})$.

Забележете, че е възможно $dist(u, v) = 0$ за $u \neq v$, защото в графа допускаме ребра с тегло 0. Затова $dist$ в случая се нарича *псевдометрика*.

Формулираме *основната задача* в тази тема: Даден е свързан граф $G = (V, E)$ с тегла по ребрата $c : E \to \mathbb{R}^{\geq 0}$, както и начален връх v_0. Да се намерят прости минимални пътища от v_0 до всички върхове на V.

В частния случай, когато $c(e)$ има една и съща стойност за всички ребра $e \in E$, задачата се решава с обхождане в ширина с начален връх v_0.

За общия случай ще разгледаме *алгоритъма на Дейкстра*.

В него съществена роля ще играят два масива d и $pred$. В масива d ще съхраняваме приближения на цените на търсените минимални пътища. На всяка итерация на основния цикъл ще намираме минимален път от v_0 до един нов връх u_0. В края на алгоритъма, масивът d ще съдържа коректните цени на минималните пътища. Тези пътища ще образуват дърво с корен v_0, което ще представим със списък на бащите в масива $pred$.

Инициализация: $d[v_0] = 0$; $d[v] = \infty$ за $v \neq v_0$; $U = \emptyset$;
Основен цикъл на алгоритъма:
докато $U \neq V$,
избираме $u_0 \notin U$, за който $d[u_0]$ е минимално;
добавяме u_0 към U: $U = U \cup \{u_0\}$;
за всички съседи u на u_0:
ако $d[u] > d[u_0] + c(\{u_0, u\})$, то
{ $d[u] = d[u_0] + c(\{u_0, u\})$; $pred[u] = u_0$; }

Считаме, че ∞ е достатъчно голяма стойност (например, по-голяма от сумата на всички ребра в графа).

Лема 8 *На всяка итерация на основния цикъл е изпълнено:* $dist(v_0, v) \leq d[v]$ *за всеки връх* $v \in V$.

Доказателство. Индукция по броя на итерациите.

База: Преди да започне изпълнението на цикъла имаме $dist(v_0, v_0) = 0 = d[v_0]$ и $dist(v_0, v) < \infty = d[v]$ за $v \neq v_0$.

Стъпка: Нека след някакъв брой итерации е изпълнено $dist(v_0, v) \leq d[v]$ за всички $v \in V$. На следващата итерация промяна в стойностите на d може да има само в $d[u]$ за някой съсед u на u_0, като новата стойност $d[u]$ е $d[u_0] + c(\{u_0, u\})$. В такъв случай:

$$dist(v_0, u) \ \leq \ dist(v_0, u_0) + c(\{u_0, u\}) \ \leq \ d[u_0] + c(\{u_0, u\}) \ = \ d[u].$$

Първото неравенство следва от (*), а второто следва от индукционното предположение. Разбира се, за стойностите на $d[v]$, които не се променят и след новата итерация имаме $dist(v_0, v) \leq d[v]$.

Лема 9 *За произволен връх* $v \in V$, *ако на някоя итерация е изпълнено* $dist(v_0, v) = d[v]$, *то това равенство се запазва и на следващите итерации до края на алгоритъма.*

Доказателство. От първата лема е изпълнено $dist(v_0, v) \leq d[v]$ на всяка итерация. Ясно е, че при актуализиране на стойността $d[v]$ тя може само да намалява. По този начин, ако $d[v]$ достигне стойност $dist(v_0, v)$ на някоя итерация тя повече няма да се променя.

Теорема 10 (коректност на алгоритъма на Дейкстра) *След завършването на основния цикъл на алгоритъма на Дейкстра, за всеки връх* $v \in V$ *е изпълнено* $dist(v_0, v) = d[v]$.

Доказателство. Достатъчно е да покажем, че на всяка итерация в момента на избор на u_0 е изпълнено $dist(v_0, u_0) = d[u_0]$. Наистина, тъй като алгоритъмът приключва при $U = V$, всеки връх трябва да бъде избран на някоя итерация. Допускаме противното и нека u_0 е *първият* избран връх, такъв че $dist(v_0, u_0) \neq d[u_0]$. Разбира се, това означава, че $u_0 \neq v_0$ и също така от

първата лема е изпълнено $dist(v_0, u_0) < d[u_0]$. Да изберем минимален път p от v_0 до u_0. В разглеждания момент при избора на u_0 имаме $u_0 \notin U$ и $v_0 \in U$, тъй като v_0 се избира на първата итерация. По този начин, p има за начало връх от U и за край връх извън U, така че съществува ребро $\{x, y\}$ от пътя p, такова че $x \in U$ и $y \notin U$. С други думи, пътят p изглежда така:

$$v_0, \ \ldots\ldots, \ \underbrace{x}_{\in U}, \ \underbrace{y}_{\notin U}, \ \ldots\ldots, \ u_0.$$

Разбира се, възможно е $v_0 = x$ или $u_0 = y$. Тъй като $x \in U$ имаме, че x е избран на предходна итерация преди u_0 и от избора на u_0, получаваме че на тази предходна итерация е изпълнено $dist(v_0, x) = d[x]$. На същата итерация $d[y]$ ще получи стойност $d(v_0, y)$, тъй като p е минимален път и

$$dist(v_0, y) \ = \ dist(v_0, x) + c(\{x, y\}) \ = \ d[x] + c(\{x, y\}).$$

От втората лема, тази стойност на $d[y]$ се запазва и по-нататък, докато се върнем на оригиналната итерация при избора на u_0. Но тогава имаме:

$$d[y] \ = \ dist(v_0, y) \ \leq \ dist(v_0, u_0) \ < \ d[u_0],$$

като за първото неравенство използваме, че цените на ребрата са неотрицателни. Достигнахме до противоречие с избора на u_0, като връх извън U с минимална стойност на d.

Следствие 11 *След изпълнението на алгоритъма, pred задава списък на бащите на дърво на минимални пътища с корен v_0.*

Доказателство. На всяка итерация, *pred* задава списък на бащите на дърво с корен v_0, което съдържа върховете v с $d[v] \neq \infty$ и освен това единственият прост път от v_0 до v в това дърво има цена $d[v]$ (забележете, че ако стойността $d[u]$ се променя на някоя итерация, то $u \notin U$ на тази итерация и затова върхът u не може да има наследници в текущото дърво). В края на алгоритъма $dist(v_0, v) = d[v]$ за всеки връх v, така че *pred* задава дърво на минимални пътища с корен v_0, което съдържа всички върхове.